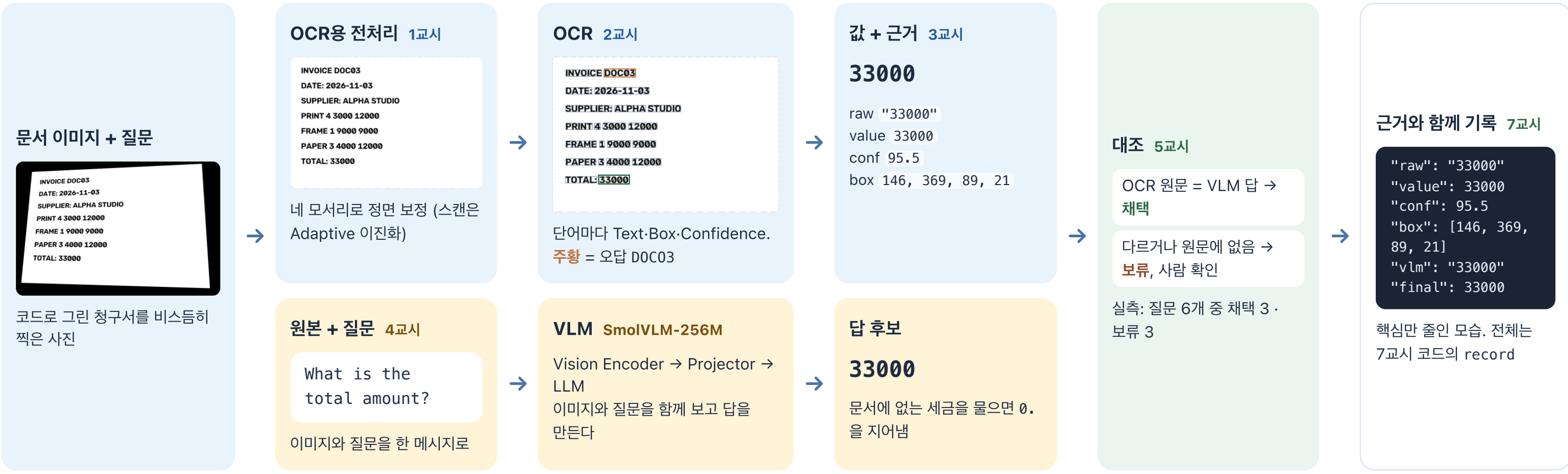
새싹 학습 노트 · Day16 · 2026.11.09 · Vision·멀티모달 AI 응용 4/4 · oneday 요약

OCR·VLM 문서 AI와 엣지 배포

오늘 한 문장 문서 이미지를 OCR이 읽기 좋게 다듬고(전처리), 단어마다 글자·위치·점수를 받아(OCR), 라벨 옆 값을 원문·좌표와 함께 JSON으로 남긴다(구조화). VLM의 답은 그 원문 근거와 맞을 때만 받아들이고, 맞지 않으면 보류해 사람이 확인한다(Hybrid). 배포는 지원되는 신경망 모델을 ONNX → TensorRT Engine으로 바꿔 대상 장치에서 검증하는 설계다.

1 한 장으로 보는 문서 AI

두 경로로 값을 얻고, 원문과 대조한 뒤, 근거와 함께 남긴다



문서 이미지 + 질문 코드로 그린 청구서를 비스듬히 찍은 사진

OCR용 전처리 1교시 네 모서리로 정면 보정 (스캔은 Adaptive 이진화)

OCR 2교시 단어마다 Text·Box·Confidence. 주황 = 오답 DOC03

값 + 근거 3교시 33000 raw "33000" value 33000 conf 95.5 box 146, 369, 89, 21

원본 + 질문 4교시 What is the total amount? 이미지와 질문을 한 메시지로

VLM SmolVLM-256M Vision Encoder → Projector → LLM 이미지와 질문을 함께 보고 답을 만든다

답 후보 33000 문서에 없는 세금을 물으면 0. 을 지어냄

대조 5교시 OCR 원문 = VLM 답 → **채택** / 다르거나 원문에 없음 → **보류**, 사람 확인. 실측: 질문 6개 중 채택 3 · 보류 3

근거와 함께 기록 7교시

```
"raw": "33000"
"value": 33000
"conf": 95.5
"box": [146, 369,
89, 21]
"vlm": "33000"
"final": 33000
```

핵심만 줄인 모습. 전체는 7교시 코드의 record

강의 PDF p.3의 구조를 따랐습니다. 사진·Box·숫자는 모두 핵심 코드를 실제로 실행한 결과입니다(Tesseract 5.5.2, HuggingFaceTB/SmolVLM-256M-Instruct).

꼭 알고 갈 개념

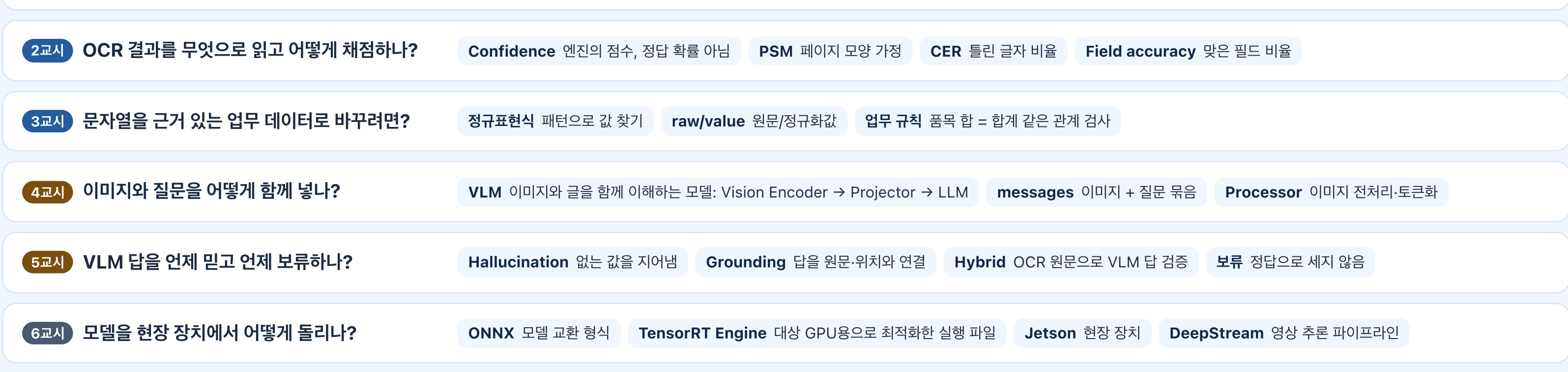
1. **OCR은 문자열만 주지 않는다.** 단어마다 Text·Box(위치)·Confidence(점수)를 준다. 이 세 가지가 뒤에서 값의 근거가 된다.
2. **VLM은 답변 "후보"를 만든다.** 이미지와 질문을 함께 이해하지만, 보이지 않는 값을 자연스럽게 지어낼 수 있다(Hallucination). 그래서 원문 근거와 대조한다(Grounding).
3. **평가 단위는 서로 다르다.** 글자 오류(CER), 필드 정답(Field accuracy), JSON 문법, 근거 일치는 각각 따로 센다. 하나가 좋다고 다른 것이 보장되지 않는다.

하루 내내 지키는 약속

1. **원본을 덮어쓰지 않는다.** 원본·전처리본·OCR 결과·최종 JSON을 나눠 저장하고, 좌표가 어느 이미지 기준인지 함께 적는다
2. **값은 근거와 함께.** `raw`(원문)·`value`(정규화값)·`bbox`·`confidence`를 묶는다. 근거가 없으면 0이나 빈 문자열이 아니라 `null`과 검토 이유를 남긴다
3. **분모와 출처를 섞지 않는다.** 같은 문서·정답·정규화 규칙에서 비교하고, 직접 실행한 결과인지 기록·제공된 결과인지 출처를 함께 적는다

2 하루 전체 지도

8교시가 네 묶음으로 이어진다



OCR 경로 이미지에서 글자와 값을 꺼낸다

- 1교시 **OCR 흐름·문서 전처리** "OCR이 읽기 좋은 이미지는?" 이진화 · Otsu · Adaptive · Perspective 보정
- 2교시 **OCR 실행·Box·품질평가** "OCR 결과를 무엇으로 읽고 어떻게 채점하나?" Confidence · PSM · CER · Field accuracy
- 3교시 **정보 추출·JSON 구조화** "문자열을 근거 있는 업무 데이터로 바꾸려면?" 정규표현식 · raw/value · 업무 규칙

VLM 경로 이미지에 묻고 답을 검증한다

- 4교시 **VLM·이미지 질의응답** "이미지와 질문을 어떻게 함께 넣나?" VLM · messages · Processor
- 5교시 **OCR·VLM 결합·평가·보안** "VLM 답을 언제 믿고 언제 보류하나?" Hallucination · Grounding · Hybrid · 보류

배포 설계 현장 장치에서 돌린다

- 6교시 **TensorRT·Jetson·DeepStream** "모델을 현장 장치에서 어떻게 돌리나?" ONNX · TensorRT Engine · Jetson · DeepStream

통합·검증 기록을 잇고 다시 실행한다

- 7교시 **OCR·VLM 미니 앱 통합** "한 문서의 처리 기록을 끝까지 잇는가?" 처리 이력 · 결합 규칙
- 8교시 **최종 검증·평가·회고** "다시 실행해도 같은 결과인가?" 재실행·대조 · 출처 구분

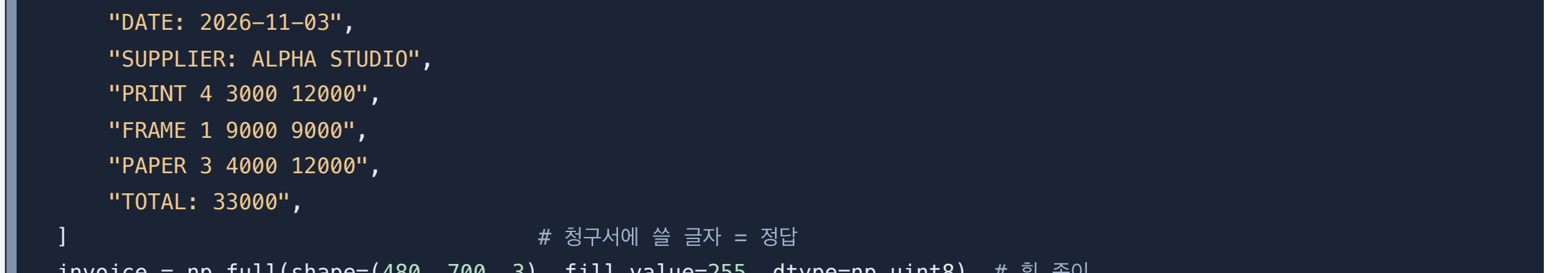
3 교시별 핵심

핵심 질문과, 꼭 알아야 할 개념의 한 줄 뜻

1교시	OCR이 읽기 좋은 이미지는?	**이진화** 글자·배경을 흑백으로 나눔 **Otsu** 문서 전체에 기준 하나 vs **Adaptive** 주변 블록마다 기준 **Perspective 보정** 비스듬한 문서를 정면으로
2교시	OCR 결과를 무엇으로 읽고 어떻게 채점하나?	**Confidence** 엔진의 점수, 정답 확률 아님 **PSM** 페이지 모양 가정 **CER** 틀린 글자 비율 **Field accuracy** 맞은 필드 비율
3교시	문자열을 근거 있는 업무 데이터로 바꾸려면?	**정규표현식** 패턴으로 값 찾기 **raw/value** 원문/정규화값 **업무 규칙** 품목 합 = 합계 같은 관계 검사
4교시	이미지와 질문을 어떻게 함께 넣나?	**VLM** 이미지와 글을 함께 이해하는 모델: Vision Encoder → Projector → LLM **messages** 이미지 + 질문 묶음 **Processor** 이미지 전처리·토큰화
5교시	VLM 답을 언제 믿고 언제 보류하나?	**Hallucination** 없는 값을 지어냄 **Grounding** 답을 원문·위치와 연결 **Hybrid** OCR 원문으로 VLM 답 검증 **보류** 정답으로 세지 않음
6교시	모델을 현장 장치에서 어떻게 돌리나?	**ONNX** 모델 교환 형식 **TensorRT Engine** 대상 GPU용으로 최적화한 실행 파일 **Jetson** 현장 장치 **DeepStream** 영상 추론 파이프라인
7교시	한 문서의 처리 기록을 끝까지 잇는가?	**처리 이력** 원문·값·근거·검토 이유를 한 기록으로 **결합 규칙** 같으면 채택, 다르면 검토
8교시	다시 실행해도 같은 결과인가?	**재실행·대조** 같은 입력·설정 → 같은 결과 **출처 구분** 직접 실행 vs 기록된 결과

4 코드로 보는 하루 흐름

공식 문서의 기본 사용법 그대로, 위에서 아래로 읽으면 이해되도록 썼습니다



- **데이터** 준비 내장 스캔 페이지 + putText로 그린 청구서 → 비스듬히 찍은 사진으로 변형
- **① 전처리** 1 스캔 페이지는 주변 블록 기준 이진화, 사진은 네 모서리로 정면 보정
- **② OCR** 2 글자 전체 + 단어별 Text·Box·점수 → 낮은 점수는 검토 목록 → CER
- **③ 구조화** 3 라벨 줄에서 값 찾기 → 원문·값·점수·Box → 품목 합 = 합계 검증
- **④ VLM 질문** 4 이미지 + 질문을 messages로 → Pipeline → 답
- **⑤ 결합** 5 VLM 답이 OCR 원문과 같으면 채택, 다르면 보류(None)
- **⑥ 배포 설계** 6 PyTorch → ONNX → 대상 장치에서 TensorRT Build (공식 명령 기록)
- **⑦ 통합 기록** 7 CER·필드·검토 이유·VLM 답·최종 값을 한 JSON으로
- **⑧ 재실행** 8 같은 코드를 다시 실행해 CER·필드·JSON이 같은지 대조

```python
import json
import re

import cv2
import jiwer
import numpy as np
import pytesseract
from skimage import data
from transformers import pipeline
```

import import OpenCV는 이미지 처리, pytesseract는 OCR, jiwer는 CER, transformers는 VLM을 맡습니다. 데이터는 scikit-image에 들어 있습니다.

```python
page = data.page()                  # scikit-image 내장 스캔 페이지 (왼쪽이 어둡다)

lines = [
    "INVOICE DOC03",
    "DATE: 2026-11-03",
    "SUPPLIER: ALPHA STUDIO",
    "PRINT 4 3000 12000",
    "FRAME 1 9000 9000",
    "PAPER 3 4000 12000",
    "TOTAL: 33000",
]                                   # 청구서에 쓸 글자 = 정답
invoice = np.full(shape=(480, 700, 3), fill_value=255, dtype=np.uint8)  # 흰 종이
for i, line in enumerate(lines):
    y = 60 + 55 * i                 # 줄마다 55픽셀씩 아래로
    cv2.putText(
        img=invoice,
        text=line,
        org=(40, y),                # 글자를 쓰기 시작할 위치 (x, y)
        fontFace=cv2.FONT_HERSHEY_SIMPLEX,
        fontScale=1,
        color=(0, 0, 0),            # 검정 (B, G, R)
        thickness=2,
    )

# 네 모서리 순서: 왼쪽 위 → 오른쪽 위 → 오른쪽 아래 → 왼쪽 아래
corners = np.float32([[60, 40], [690, 10], [720, 470], [20, 450]])  # 사진 속 문서
front = np.float32([[0, 0], [699, 0], [699, 479], [0, 479]])        # 정면 문서
tilt = cv2.getPerspectiveTransform(src=front, dst=corners)   # 정면 → 비스듬
photo = cv2.warpPerspective(src=invoice, M=tilt, dsize=(760, 500))  # 비스듬한 사진
```

준비 데이터 왼쪽이 어두운 스캔 페이지를 불러오고, 청구서 7줄을 그린 뒤 **일부러 비스듬히 찍은 사진**으로 바꿉니다. 그린 글자 `lines`가 곧 정답입니다.

```python
binary = cv2.adaptiveThreshold(
    src=page,
    maxValue=255,                   # 기준보다 밝은 픽셀 → 흰색(255)
    adaptiveMethod=cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  # 주변의 가중 평균으로 기준
    thresholdType=cv2.THRESH_BINARY,
    blockSize=31,                   # 주변 31×31 칸을 보고 기준을 정한다
    C=11,                           # 그 평균에서 11을 뺀 값이 기준
)
M = cv2.getPerspectiveTransform(src=corners, dst=front)     # 사진 → 정면
warped = cv2.warpPerspective(src=photo, M=M, dsize=(700, 480))  # (너비, 높이)
cv2.imwrite(filename="invoice.png", img=warped)  # 원본 photo는 그대로 둔다
```

1교시 ① 전처리 스캔 페이지는 **주변 31×31 블록마다 기준**을 잡아 흑백으로 나눕니다. 사진은 네 모서리를 정면 네 모서리로 옮겨 펴고, 원본은 그대로 둔 채 보정본을 저장합니다.

```python
text = pytesseract.image_to_string(image=warped, config="--psm 6")
words = pytesseract.image_to_data(
    image=warped,
    config="--psm 6",               # 페이지 모양 가정: 글 블록 하나
    output_type=pytesseract.Output.DATAFRAME,  # 결과를 표(DataFrame)로
)
words = words[words.conf >= 0]          # 단어가 있는 행만 남긴다
review = words[words.conf < 50]         # 점수가 낮은 단어 → 지우지 않고 검토 목록
truth = " ".join(lines)                 # 정답 글자
read = " ".join(text.split())           # OCR 글자 (줄바꿈·공백을 한 칸으로)
score = jiwer.cer(reference=truth, hypothesis=read)  # 글자 오류율 CER
```

2교시 ② OCR `image_to_string`은 글자 전체, `image_to_data`는 단어마다 **Text·Box·Confidence** 표를 줍니다. 낮은 점수는 지우지 않고 검토 목록으로 두고, 정답과 비교해 CER을 잽니다.

```python
match = re.search(
    pattern=r"^TOTAL: (\d+)$",      # "TOTAL: " 뒤의 숫자
    string=text,
    flags=re.MULTILINE,             # ^와 $를 줄마다 적용
)
raw = match.group(1)                    # "33000" : OCR이 읽은 원문
row = words[words.text == raw].iloc[0]  # 그 숫자가 적힌 단어 (첫 번째 행)
total = {
    "raw": raw,
    "value": int(raw),                  # 33000 : 숫자로 바꾼 값
    "conf": float(row.conf),
    "box": [int(row.left), int(row.top), int(row.width), int(row.height)],
}
amounts = re.findall(
    pattern=r"^[A-Z]+ \d+ \d+ (\d+)$",  # 품목 줄의 마지막 숫자 = 금액
    string=text,
    flags=re.MULTILINE,
)
reasons = []
if sum(int(a) for a in amounts) != total["value"]:  # 품목 합 ≠ 합계면 검토
    reasons.append("total_mismatch")
```

3교시 ③ 구조화 `TOTAL:` 줄에서 괄호 안 숫자만 꺼내 **원문(raw)과 숫자 값(value)**을 함께 남기고, 그 단어의 점수·위치를 근거로 붙입니다. 품목 합이 합계와 다르면 검토 이유를 기록합니다.

```python
pipe = pipeline(
    task="image-text-to-text",
    model="HuggingFaceTB/SmolVLM-256M-Instruct",
)
messages = [
    {"role": "user", "content": [
        {"type": "image", "path": "invoice.png"},
        {"type": "text", "text": "What is the total amount?"},
    ]},
]
out = pipe(
    text=messages,
    max_new_tokens=128,             # 답의 최대 길이 (토큰 수)
    do_sample=False,                # 같은 입력 → 같은 답
    return_full_text=False,         # 질문은 빼고 답만 받는다
)
answer = out[0]["generated_text"].strip()  # VLM의 답 (문자열)
```

4교시 ④ VLM 질문 이미지와 질문을 **한 메시지**로 묶어 묻습니다. `do_sample=False`면 같은 입력에 같은 답, `return_full_text=False`면 새 답만 받습니다.

```python
if answer == total["raw"]:
    final = total["value"]              # 두 결과가 같다 → 채택
else:
    final = None                        # 다르다 → 보류, 사람이 확인
```

5교시 ⑤ 결합 VLM 답이 **OCR 원문과 같을 때만 채택**합니다. 다르면 None(보류)으로 두고 사람이 확인합니다. 보류는 정답으로 세지 않습니다.

```python
# torch.onnx.export(model=model, args=(x,), f="model.onnx")  # PyTorch → ONNX
# trtexec --onnx=model.onnx --saveEngine=model.engine   # 대상 장치에서 Build
```

6교시 ⑥ 배포 설계 장비가 필요해 실행하지 않고 공식 명령만 남깁니다. PyTorch → ONNX로 내보내고, **실제로 돌릴 장치에서** TensorRT Engine을 Build합니다.

```python
record = {
    "document_id": "doc03",
    "cer": score,
    "total_amount": total,
    "review_reasons": reasons,
    "vlm_answer": answer,
    "final": final,
}
with open(file="doc03.json", mode="w", encoding="utf-8") as f:
    json.dump(obj=record, fp=f, ensure_ascii=False, indent=2)
```

7교시 ⑦ 통합 기록 CER·합계(원문·값·점수·위치)·검토 이유·VLM 답·최종 값을 **한 JSON**으로 남깁니다. `ensure_ascii=False`면 원문 글자가 그대로 저장됩니다.

데이터는 따로 받지 않습니다. 스캔 페이지는 `skimage.data.page()`에 들어 있고, 청구서는 코드가 직접 그립니다(그린 글자 = 정답). 위 코드는 그대로 실행되며 Colab 노트북으로도 열 수 있습니다. 8교시는 같은 코드를 다시 실행해 CER·필드·JSON을 대조합니다.

꼭 알고 갈 함수 · 사용 가이드

오늘 흐름에서만 의미가 있는 함수입니다. 짝으로 쓰는 함수는 한 장으로 묶었습니다.

1교시 `cv2.adaptiveThreshold`

- **용도** 조명·그림자가 고르지 않은 문서를 흑백으로 나눌 때. 조명이 고르면 `cv2.threshold`(Otsu)로 충분하다
- **사용법** `cv2.adaptiveThreshold(src=gray, maxValue=255, adaptiveMethod=cv2.ADAPTIVE_THRESH_GAUSSIAN_C, thresholdType=cv2.THRESH_BINARY, blockSize=31, C=11)`. 컬러 이미지는 먼저 `cv2.cvtColor(src=img, code=cv2.COLOR_BGR2GRAY)`로 Gray로 바꾼다
- **확인할 점** blockSize(3 이상 홀수)와 C를 몇 개 바꿔 OCR CER이 가장 낮은 값을 고른다

1교시 `cv2.getPerspectiveTransform → cv2.warpPerspective`

- **용도** 비스듬히 찍은 문서를 정면으로 펼 때
- **사용법** 사진 속 네 모서리와 정면 네 모서리를 **같은 순서**의 `np.float32` 목록으로 → `M = getPerspectiveTransform(src=사진_모서리, dst=정면_모서리)` → `warpPerspective(src=img, M=M, dsize=(너비, 높이))`
- **확인할 점** 결과 이미지를 눈으로 확인한다. 이후 OCR Box는 보정본 기준 좌표다

2교시 `pytesseract.image_to_string · image_to_data`

- **용도** 글자 전체가 필요하면 `image_to_string`(값 찾기용), 단어별 위치·점수가 필요하면 `image_to_data`(근거·검토용)
- **사용법** `image_to_data(image=img, config="--psm 6", output_type=pytesseract.Output.DATAFRAME)` → `text · conf · left · top · width · height` 표
- **확인할 점** PSM은 입력 모양에 맞춘다: 문서 한 장 6, 한 줄 Crop 7, 한 단어 8. `conf`가 낮은 단어는 지우지 않고 원본에서 눈으로 확인한다

2교시 `jiwer.cer`

- **용도** 전처리·PSM 후보 중 무엇이 나은지 숫자로 비교할 때
- **사용법** `jiwer.cer(reference=정답, hypothesis=OCR결과)`. 두 문자열 모두 공백·줄바꿈을 같은 규칙으로 정리한 뒤 넣는다
- **확인할 점** CER과 필드 정답을 함께 본다. CER이 낮아도 필드가 틀릴 수 있다

3교시 `re.search → .group(1)`

- **용도** 라벨 뒤에 오는 값(합계·날짜)을 꺼낼 때
- **사용법** `re.search(pattern=r"^TOTAL: (\d+)$", string=text, flags=re.MULTILINE)` → `.group(1)`. 꺼낼 부분만 괄호로 감싸고, 여러 줄 글이면 `re.MULTILINE`
- **확인할 점** 못 찾으면 `None`이 온다. `.group()` 전에 확인하고, 못 찾은 필드는 검토 목록에 남긴다

4교시 `pipeline → pipe(text=messages, ...)`

- **용도** 이미지에 질문하고 답을 받을 때
- **사용법** `pipe = pipeline(task="image-text-to-text", model=모델ID)` → `messages`에 image 항목과 text 항목을 함께 → `pipe(text=messages, max_new_tokens=128, do_sample=False, return_full_text=False)` → `out[0]["generated_text"]`
- **확인할 점** `do_sample=False`로 같은 답을 재현하고, `max_new_tokens`는 답이 잘리지 않을 만큼 준다. 답은 그대로 쓰지 않고 OCR 원문과 대조한다

6교시 `torch.onnx.export → trtexec`

- **용도** 학습한 신경망을 Jetson·GPU에서 빠르게 돌릴 때
- **사용법** `torch.onnx.export(model=model, args=(x,), f="model.onnx")` → 원본과 결과 비교 → 대상 장치에서 `trtexec --onnx=model.onnx --saveEngine=model.engine`
- **확인할 점** 실제로 넣을 batch·입력 크기로 실행해 본다(바뀌는 축은 `dynamic_shapes`). Engine은 만든 장치·버전에서만 쓴다

7교시 `json.dump`

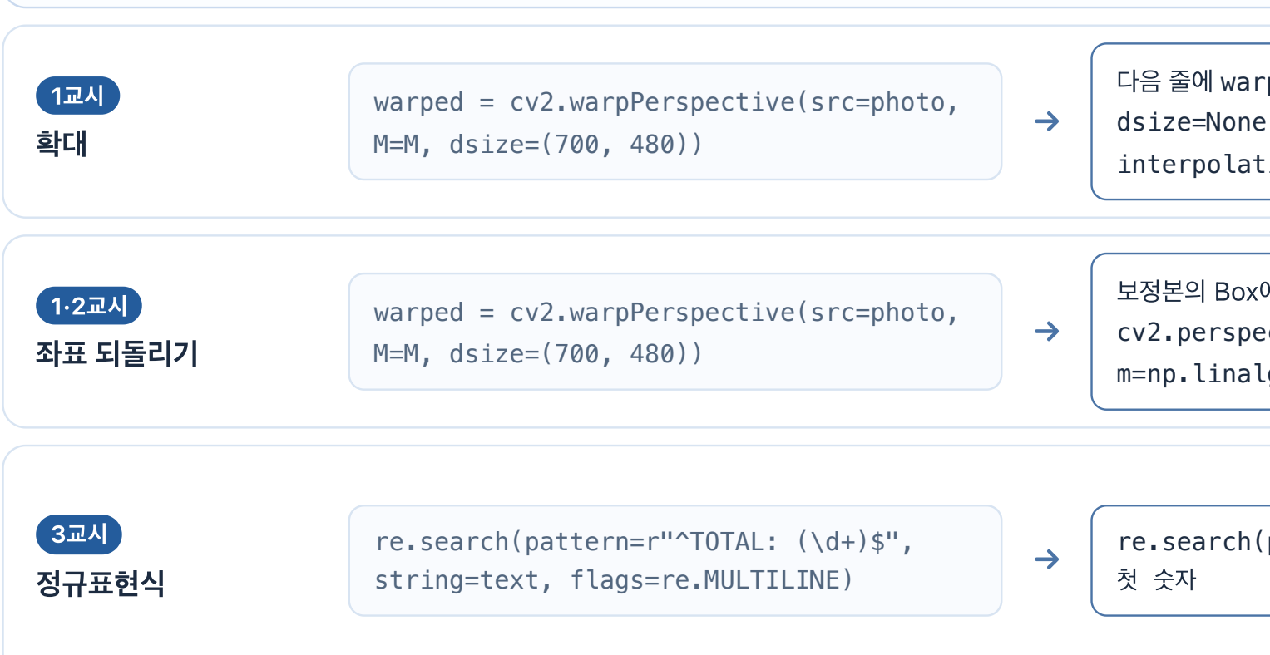
- **용도** 한 문서의 결과(원문·값·근거·검토 이유)를 파일로 남길 때
- **사용법** `with open(file=path, mode="w", encoding="utf-8") as f:` 안에서 `json.dump(obj=record, fp=f, ensure_ascii=False, indent=2)`
- **확인할 점** numpy 숫자는 `int()` · `float()`로 바꿔 넣는다

코드 속 파라미터 · 어디를 고치면 무엇이 달라지나

왼쪽은 위 핵심 코드 그대로, 가운데는 표시한 부분만 고친 코드입니다. 고친 코드로 다시 실행한 결과를 오른쪽에 적었습니다. **꼭 기억** 표시가 붙은 7개는 오늘 반드시 가져갈 결과입니다.

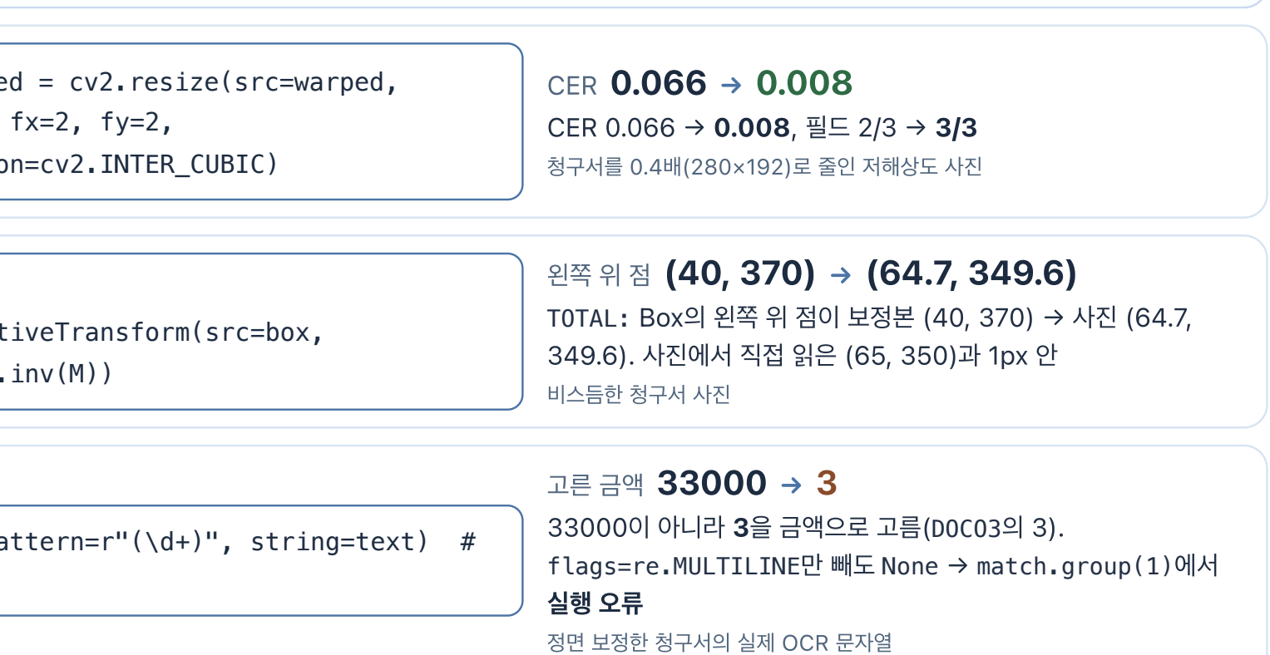
숫자·옵션만 바꾸기 줄은 그대로 두고 값만 고쳤습니다

	위 코드 (핵심 코드 그대로)	고친 코드 (이 부분만 바꿔 다시 실행)	결과
1교시 **blockSize**	`blockSize=31,`	`blockSize=3,`	CER **0.057 → 0.137** CER 0.057 → **0.137**. 15는 0.050, 101은 0.084. 짝수 30은 **실행 오류** 내장 스캔 페이지 · 이진화한 페이지를 PSM 6으로 OCR **꼭 기억** 주변을 몇 칸 보고 기준을 정할지다. 너무 작으면 획 안쪽까지 배경으로 본다. 3 이상 홀수만 된다
1교시 **C**	`C=11,`	`C=0,`	CER **0.057 → 0.388** CER 0.057 → **0.388**. 30은 **0.020**(더 좋아짐), 60은 0.221 내장 스캔 페이지
2교시 **PSM**	`config="--psm 6"`	`config="--psm 7"  # 한 줄로 가정`	CER **0.008 → 1.0** 청구서 CER 0.008 → **1.0**(빈 문자열). PSM 8(한 단어)은 "a". 합계 한 줄만 잘라 넣으면 PSM 7에서 `TOTAL: 33000` 정확 정면 보정한 청구서 / 합계 줄 Crop **꼭 기억** PSM은 "페이지가 어떻게 생겼다"는 가정이다. 입력 범위에 맞춰 고르고, 입력 범위가 다른 결과끼리 CER로 순위를 매기지 않는다
2교시 **confidence 기준**	`review = words[words.conf < 50]`	`review = words[words.conf < 90]`	검토 목록 **0 → 1** 검토 목록 0개 → **1개**. 유일한 오답 `DOC03`(숫자 0을 글자 O로 읽음)의 점수는 **65.6**이라 50 기준은 통과. 맞는 단어 20개는 모두 92.8 이상 정면 보정한 청구서 · 단어 21개. 실습 문서 5장에서도 오답 5개가 34~80점 **꼭 기억** confidence는 정답 확률이 아니라 검토 순서다. 기준을 정하기 전에 오답의 점수 분포를 본다. 낮은 점수를 지우지 않고 사람이 보도록 남긴다
4교시 **max_new_tokens**	`max_new_tokens=128,`	`max_new_tokens=3,`	답 **33000 → 33** 답 `33000` → **33**. 오류 없이 잘린 숫자가 나옴. 5교시 대조에서 OCR 원문과 달라 보류됨 정면 보정한 청구서 · "What is the total amount?"



blockSize와 C에 따른 CER (내장 스캔 페이지 · 낮을수록 좋음)

blockSize는 3 이상 홀수만 됩니다(30을 넣으면 실행 오류). C는 30에서 가장 좋았습니다. 알맞은 값은 문서마다 다르니 CER로 고릅니다.



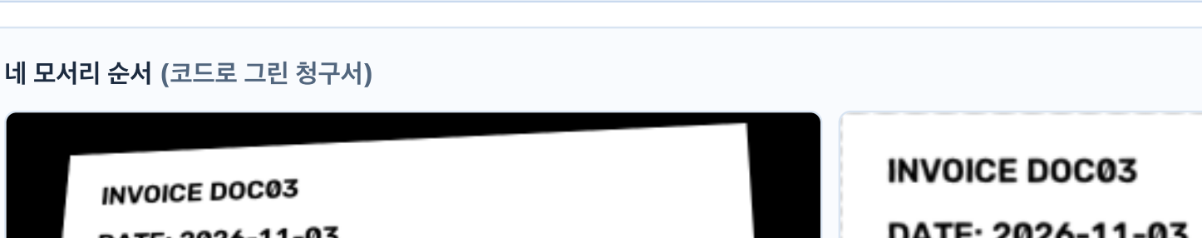
단어별 confidence (정면 보정한 청구서)

유일한 오답은 숫자 0을 글자 O로 읽은 `DOC03`입니다. 맞는 단어 20개는 모두 92.8점 이상. 기준은 오답의 점수 분포를 보고 정합니다.

코드 한 줄 넣기·바꾸기 줄을 더하거나 다른 함수·문장으로 바꿨습니다

	위 코드 (핵심 코드 그대로)	고친 코드 (이 부분만 바꿔 다시 실행)	결과
1교시 **이진화 방법**	`binary = cv2.adaptiveThreshold(src=page, ...)`	`_, binary = cv2.threshold(src=page, thresh=0, maxval=255, type=cv2.THRESH_BINARY + cv2.THRESH_OTSU)`	CER **0.057 → 0.361** CER 0.057 → **0.361**. Otsu가 페이지 전체에 기준 157 하나를 써서 어두운 왼쪽 글자가 묻힘. 이진화 없이 Gray 그대로는 0.324 내장 스캔 페이지 **꼭 기억** 조명이 고르지 않으면 전역 기준 하나로는 안 된다. 주변 블록마다 기준을 따로 잡는 것이 Adaptive다. C 같은 값은 문서마다 알맞은 값이 다르니 CER로 고른다
1교시 **확대**	`warped = cv2.warpPerspective(src=photo, M=M, dsize=(700, 480))`	다음 줄에 `warped = cv2.resize(src=warped, dsize=None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)`	CER **0.066 → 0.008** CER 0.066 → **0.008**, 필드 2/3 → **3/3** 청구서를 0.4배(280×192)로 줄인 저해상도 사진
1·2교시 **좌표 되돌리기**	`warped = cv2.warpPerspective(src=photo, M=M, dsize=(700, 480))`	보정본의 Box에 `cv2.perspectiveTransform(src=box, m=np.linalg.inv(M))`	왼쪽 위 점 **(40, 370) → (64.7, 349.6)** `TOTAL:` Box의 왼쪽 위 점이 보정본 (40, 370) → 사진 (64.7, 349.6). 사진에서 직접 읽은 (65, 350)과 1px 안 비스듬한 청구서 사진
3교시 **정규표현식**	`re.search(pattern=r"^TOTAL: (\d+)$", string=text, flags=re.MULTILINE)`	`re.search(pattern=r"(\d+)", string=text)  # 첫 숫자`	고른 금액 **33000 → 3** 33000이 아니라 **3**을 금액으로 고름(`DOC03`의 3). `flags=re.MULTILINE`만 빼도 None → `match.group(1)`에서 **실행 오류** 정면 보정한 청구서의 실제 OCR 문자열
4교시 **질문**	`{"type": "text", "text": "What is the total amount?"}`	`"What is the tax amount?"` 등 **문서에 없는 값**을 질문	답 **합계 33000 → 세금 0.** 세금 → `0.`, 할인액 → `33000`(합계를 그대로), 공급자 전화번호 → `ALPHA STUDIO`. 질문 끝에 "없으면 null이라고 답해"를 붙여도 `0.` 정면 보정한 청구서 · 문서에는 세금·할인·전화번호가 없음 **꼭 기억** VLM은 모르는 값도 자연스럽게 지어낸다(Hallucination). 프롬프트만으로는 막지 못하니, 답을 OCR 원문 같은 근거와 대조한다

이진화 방법 (내장 스캔 페이지 · PSM 6)



원본 Gray CER 0.324 · **Otsu (기준 157)** CER 0.361 · **Adaptive (31, 11)** CER 0.057

Otsu는 페이지 전체에 기준 하나를 써서 어두운 왼쪽 글자가 검게 묻힙니다. Adaptive는 주변마다 기준을 따로 잡아 글자를 살립니다.

실제 VLM에 물어본 결과 (SmolVLM-256M-Instruct · do_sample=False)

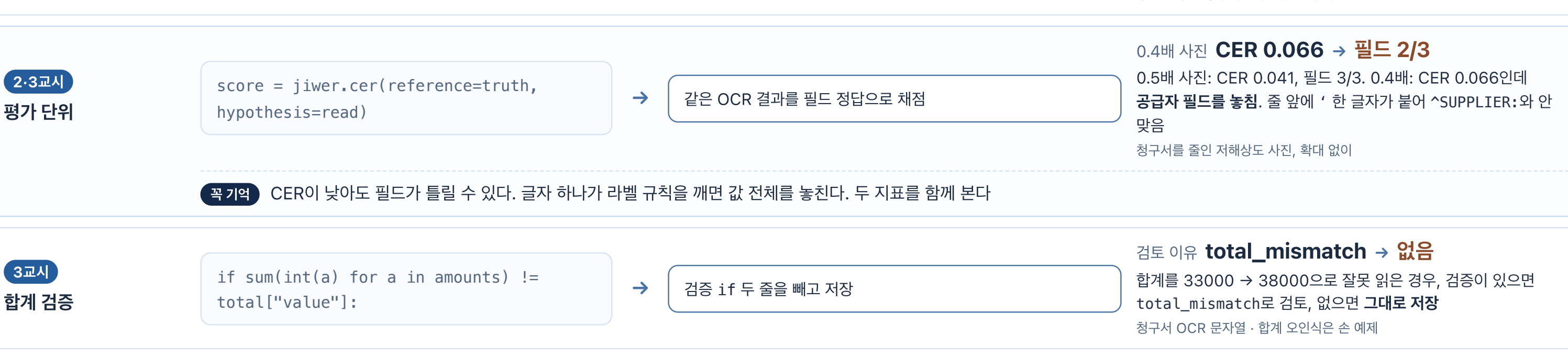
질문	VLM 답	OCR 원문	대조
합계	`33000`	`33000`	채택
공급자	`ALPHA STUDIO`	`ALPHA STUDIO`	채택
발행일	`2026-11-03`	`2026-11-03`	채택
세금	`0.`	문서에 없음	보류
할인액	`33000`	문서에 없음	보류
전화번호	`ALPHA STUDIO`	문서에 없음	보류

문서에 없는 값도 자연스럽게 지어냅니다. "없으면 null이라고 답해"를 붙여도 세금은 `0.`. "JSON으로 답해"에도 `33000`만 답해 형식을 지키지 않았습니다. 그래서 답을 OCR 원문과 대조합니다.

순서만 바꾸기 같은 값을 다른 자리에 넣었습니다 (흔한 실수)

	위 코드 (핵심 코드 그대로)	고친 코드 (이 부분만 바꿔 다시 실행)	결과
1교시 **모서리 순서**	`corners = np.float32([[60, 40], [690, 10], [720, 470], [20, 450]])`	`corners = np.float32([[690, 10], [720, 470], [20, 450], [60, 40]])  # 시작점이 오른쪽 위`	CER **0.008 → 1.254** CER 0.008 → **1.254**, 필드 3/3 → **0/3**. 문서가 90도 돌아가 찌그러짐(잘못 끼어든 글자 때문에 CER이 1을 넘음). 두 점만 맞바꾸면 사각형이 꼬여 화면 대부분이 검게 나옴(CER 1.0) 비스듬한 청구서 사진 **꼭 기억** `src` (사진 속 모서리)와 `dst` (정면 모서리)의 네 점은 같은 순서(왼쪽 위 → 오른쪽 위 → 오른쪽 아래 → 왼쪽 아래)여야 한다. 보정 결과를 눈으로 확인한다

네 모서리 순서 (코드로 그린 청구서)



INVOICE DOC03
DATE: 2026-11-03
SUPPLIER: ALPHA STUDIO
PRINT 4 3000 12000
FRAME 1 9000 9000
PAPER 3 4000 12000
TOTAL: 33000

비스듬한 사진 CER 0.008 · **정면 보정** CER 0.008 · 필드 3/3 · **시작점이 오른쪽 위** CER 1.254 · 필드 0/3

이 정도 기울기는 Tesseract가 보정 없이도 읽습니다. 하지만 시작점을 잘못 잡으면 문서가 90도 돌아가 찌그러집니다. 두 점만 맞바꾸면 사각형이 꼬여 화면의 99%가 검게 나옵니다(CER 1.0).

모델은 그대로, 판정하고 세는 방법만 바꾸기 OCR·VLM 출력은 같고 판정 규칙만 바꿨습니다

	위 코드 (핵심 코드 그대로)	고친 코드 (이 부분만 바꿔 다시 실행)	결과
5교시 **대조**	`if answer == total["raw"]:` → 채택, 아니면 `None`	`final = answer  # 대조 없이 채택`	할인액 **보류 → 33000** 할인액 질문의 답 `33000`이 그대로 **할인액으로 저장**됨. 대조하면 OCR에 할인 줄이 없어 보류 정면 보정한 청구서 · 위 "질문" 행의 답
2·3교시 **평가 단위**	`score = jiwer.cer(reference=truth, hypothesis=read)`	같은 OCR 결과를 필드 정답으로 채점	0.4배 사진 **CER 0.066 → 필드 2/3** 0.5배 사진: CER 0.041, 필드 3/3. 0.4배: CER 0.066인데 **공급자 필드를 놓침**. 줄 앞에 ‘ 한 글자가 붙어 `^SUPPLIER:`와 안 맞음 청구서를 줄인 저해상도 사진, 확대 없이 **꼭 기억** CER이 낮아도 필드가 틀릴 수 있다. 글자 하나가 라벨 규칙을 깨면 값 전체를 놓친다. 두 지표를 함께 본다
3교시 **합계 검증**	`if sum(int(a) for a in amounts) != total["value"]:`	검증 `if` 두 줄을 빼고 저장	검토 이유 **total_mismatch → 없음** 합계를 33000 → 38000으로 잘못 읽은 경우, 검증이 있으면 `total_mismatch`로 검토, 없으면 **그대로 저장** 청구서 OCR 문자열 · 합계 오인식은 손 예제

CER과 필드 정답은 다른 것을 잰다 (청구서를 줄인 저해상도 사진)

사진	CER	필드	놓친 필드
0.5배 (350×240)	0.041	3/3	없음
0.4배 (280×192)	0.066	2/3	공급자
0.4배 → 2배 확대	0.008	3/3	없음

0.4배에서 OCR이 `‘SUPPLIER: ALPHA STUDIO`처럼 줄 앞에 글자 하나를 붙여 `^SUPPLIER:` 규칙과 안 맞았습니다. CER은 6.6%뿐인데 필드 하나를 통째로 놓쳤습니다.

출처

NVIDIA AI 코어 엔지니어 2기 · Day16 Vision·멀티모달 AI 응용 4/4 · 강의 PDF `Day16_20261109.pdf`(101쪽). 코드 문법: OpenCV Image Thresholding·Geometric Transformations, pytesseract, jiwer, Hugging Face Transformers(image-text-to-text), PyTorch ONNX, TensorRT trtexec.

그림과 숫자는 핵심 코드와 같은 데이터(scikit-image 내장 스캔 페이지, 코드로 그린 청구서)로 직접 실행한 결과입니다. Tesseract 5.5.2, OpenCV 5.0.0, scikit-image 0.26.0, transformers 4.57.6, torch 2.14.0, VLM HuggingFaceTB/SmolVLM-256M-Instruct(do_sample=False). 스캔 페이지의 CER은 손으로 옮긴 정답 7줄 기준입니다.

수업 실습의 기본 경로는 강사가 미리 써 둔 VLM 응답이라 실습 결과와 VLM 답이 다를 수 있습니다. TensorRT·Jetson·DeepStream은 실행하지 않은 설계입니다. 합성 영어 문서라 실제 한국어 문서·손글씨 성능을 뜻하지 않습니다.